



UNIVERSITEIT VAN PRETORIA
UNIVERSITY OF PRETORIA
YUNIBESITHI YA PRETORIA

Faculty of Engineering, Built Environment and
Information Technology

Department of Electrical, Electronic and Computer Engineering

Control Systems - EBB 320

Practical 3: PID Controller Design

Developed by:

Heinz Mittermaier

Albertus Richter

Edited by:

Keshin Bheema

Jean Otto

Phillip Stiemie

2025

Contents

1	Introduction	3
2	Background	3
3	System Used	3
4	Controller Design	4
4.1	Design via Root Locus	4
4.2	Design via Frequency Response	4
5	System Simulation	4
5.1	Reference tracking simulation	4
5.2	Disturbance rejection simulation	4
6	Physical Implementation	5
7	Demonstration	6
8	Report	6
9	References	7

Acronyms

MCU	microcontroller unit
MIMO	multiple-input, multiple-output
PID	Proportional Integral and Derivative

1 INTRODUCTION

Proportional Integral and Derivative (PID) control is renowned for its versatility and ability to handle a wide range of control scenarios. From temperature control in industrial processes to quadcopter stabilisation and robotic motion control, PID controllers have proven their worth in countless applications. Understanding the principles behind PID control and mastering the design and tuning techniques are essential for achieving accurate and robust control.

In this practical, you will take a closer look at the PID design techniques covered in EBB320. We will then use those techniques to design a decoupled multiple-input, multiple-output (MIMO) PID controller. The MIMO system from Practical 1 contains transfer functions for G_{12} as well as G_{21} which means that the real world system is in fact not decoupled as there is heat transfer between transistor 1 and temperature sensor 2 as well as between transistor 2 and temperature sensor 1. This means that close attention will need to be paid to the controller design and implementations to ensure the physical controller can be implemented successfully.

2 BACKGROUND

Knowledge of the following topics is recommended for the completion of this practical:

- All knowledge obtained in Practical 1 and Practical 2.
- PID design via Root Locus.
- PID design via Frequency Response.

Further research on the above and other topics may be required, such as in [1]. Note, you **are not required** to write a theoretical background section in your report. However, you **are required** to show thorough understanding of these topics in your design, discussion and analysis of the results.

3 SYSTEM USED

From Practical 1, each group should have a MIMO transfer function matrix as follows:

$$\begin{aligned} \mathbf{Y}(s) &= [\mathbf{G_P}(s)] \mathbf{U}(s) \\ \begin{bmatrix} Y_1 \\ Y_2 \end{bmatrix} &= \begin{bmatrix} G_{11} & G_{12} \\ G_{21} & G_{22} \end{bmatrix} \begin{bmatrix} U_1 \\ U_2 \end{bmatrix} \end{aligned} \quad (1)$$

For the controller design, the system should be reduced to a decoupled MIMO system:

$$\begin{aligned} \mathbf{Y}(s) &= [\mathbf{G_P}(s)] \mathbf{U}(s) \\ \begin{bmatrix} Y_1 \\ Y_2 \end{bmatrix} &= \begin{bmatrix} G_{11} & 0 \\ 0 & G_{22} \end{bmatrix} \begin{bmatrix} U_1 \\ U_2 \end{bmatrix} \end{aligned} \quad (2)$$

The decoupled system in Equation 2 should be used to simulate the undisturbed system while the matrix in Equation 1 should be used to simulate the disturbed simulation.

4 CONTROLLER DESIGN

The following specifications should be used to design all controllers

- 10% overshoot.
- Threefold reduction in rise time.
- Zero steady-state error for a step input.

4.1 DESIGN VIA ROOT LOCUS

Design two PID controllers using Root Locus techniques: $C_{RL_{11}}$ and $C_{RL_{22}}$ for G_{11} and G_{22} respectively.

4.2 DESIGN VIA FREQUENCY RESPONSE

Design two PID controllers using Frequency Response techniques: $C_{FR_{11}}$ and $C_{FR_{22}}$ for G_{11} and G_{22} respectively.

5 SYSTEM SIMULATION

5.1 REFERENCE TRACKING SIMULATION

Using Simulink, simulate **all four controllers** applied to the system in the absence of disturbances. Choose feasible reference trajectories for the simulations. Include figures/graphs of your Simulink model, reference trajectories, system inputs, and system outputs.

Use the following image as a reference for your Simulink model:

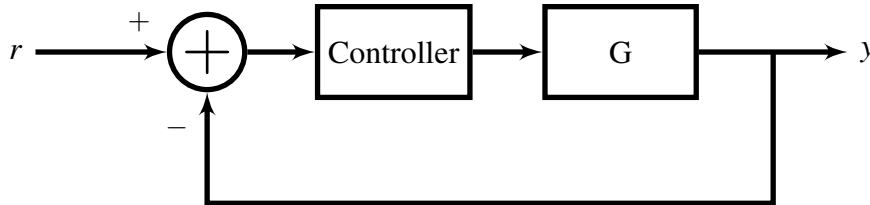


Figure 1: Simulation model for reference tracking.

In Figure 1, G can be replaced with G_{11} or G_{22} and the Controller can be replaced with the relevant controller.

5.2 DISTURBANCE REJECTION SIMULATION

Using Simulink, simulate **all four controllers** applied to the full system, now in the presence of disturbances. Choose feasible reference trajectories (i.e. setpoints) for the simulations (consider testing the reference trajectories in Table 1). Include figures/graphs of your Simulink model, reference trajectories, system inputs and system outputs.

Use the following image as a reference for your Simulink model:

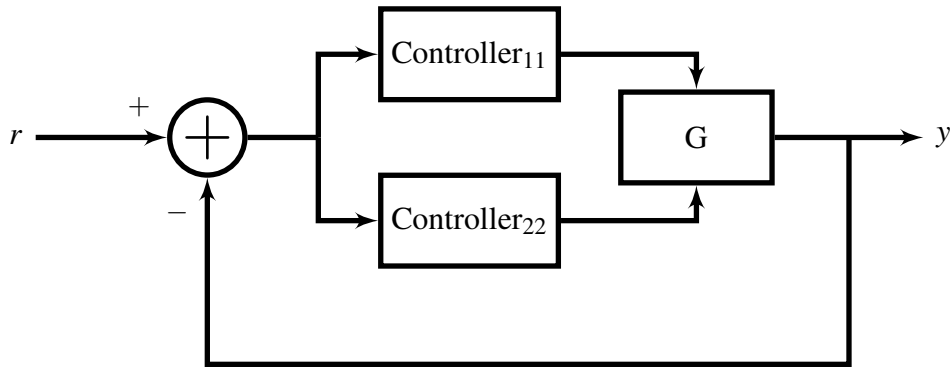


Figure 2: Simulation model for disturbance rejection.

In Figure 2, G is the transfer function matrix in Equation 1, y consists of both system outputs and r includes a reference trajectory for each temperature sensor.

6 PHYSICAL IMPLEMENTATION

For the physical implementation, you will implement both sets of PIDs designed. Note that no PID libraries may be used to implement your controller and that your PID controller should be coded as a function, where your error signal is the input to the function and the control signal is returned from the function. Here are some examples of implementing functions in MATLAB and Simulink [2]–[4].

While your controller is running, your data acquisition should be able to plot live graphs of the data gathered by your system as follows:

- Figure 1: Transistor 1 reference and temperature reading ($^{\circ}\text{C}$) vs time.
- Figure 2: Transistor 2 reference and temperature reading ($^{\circ}\text{C}$) vs time.
- Figure 3: Transistor 1 system input (i.e. manipulated variable) versus time.
- Figure 4: Transistor 2 system input versus time.

All graphs must be a function of time and not samples (i.e., the x -axis must be in seconds).

The system should have an input (either a soft input through serial communication or a physical button connected to the microcontroller unit (MCU)). The input should allow you to switch between the following states of your system (i.e., when the button is pressed, move to the next state):

State	Reference 1 ($^{\circ}\text{C}$)	Reference 2 ($^{\circ}\text{C}$)
State 1	35	35
State 2	60	35
State 3	60	50

Table 1: Required reference value states for the demonstration.

7 DEMONSTRATION

- You will only demonstrate the pair of controllers designed via the frequency response.
- Your demonstrator must be able to cycle through the various states in Table 1 using either the hardware button or software interface designed.
- Live graphs of the various reference and system inputs should be shown as described in Section 6.

Demonstration Criteria:	Marks:
PID implementation	5
Rise time	5
Percentage overshoot	5
Reference trajectories	5
Total:	20

Table 2: Demonstration mark breakdown.

Please see the Practicals tab on ClickUP for the relevant demonstration date.

8 REPORT

1. Design & Simulation

- Show the process transfer function matrix used as derived in Practical 1.
- Show the Simulink simulation models for reference tracking and disturbance rejection.
- Show the design procedure for the 4 controllers. Include the following:
 - Controller parameters.
 - Open-loop vs closed-loop step response graphs.
 - Open-loop vs closed-loop step response information (like overshoot, rise time and fall time).
- Discuss the simulation results. Compare and contrast the controller designs based on the simulation results.

2. Results & Discussion

- Show the measured results for all 4 implemented PID controllers.
- Discuss your results in relation to the simulated results. Explain any differences observed.

3. References

4. Appendix

- All code (MATLAB and embedded) should be included.

Report Section:	Mark:
Design	10
Simulation - results & discussion	15
Physical results & discussion	20
Formatting	5
Total:	50

Table 3: Report mark breakdown.

Please see the Practicals tab on ClickUP for the relevant report deadline. **Please submit your report to both the AMS and Turnitin with the cover page included in both submissions.** You do not need to submit a Turnitin receipt to the AMS.

9 REFERENCES

- [1] D. E. Seborg, T. F. Edgar, D. A. Mellichamp, and F. J. Doyle III, *Process Dynamics and Control*, 4th ed. Hoboken, NJ: Wiley, 2016, ISBN: 978-1-119-28591-5.
- [2] Mathworks, *Implement MATLAB Functions in Simulink with MATLAB Function Blocks - MATLAB & Simulink*. [Online]. Available: <https://www.mathworks.com/help/simulink/ug/what-is-a-matlab-function-block.html>.
- [3] *Create, edit, and debug MATLAB code in MATLAB Function blocks - MATLAB*. [Online]. Available: <https://www.mathworks.com/help/simulink/slref/matlabfunctionblockeditor.html>.
- [4] Mathworks, *Include MATLAB code in models that generate embeddable C code - Simulink*. [Online]. Available: <https://www.mathworks.com/help/simulink/slref/matlabfunction.html>.
- [5] Mathworks, *Tune PID controllers - MATLAB*. [Online]. Available: <https://www.mathworks.com/help/control/ref/pidtuner-app.html>.